

Ontology for Payments and Merchant Applications

A Context Graph Architecture for PesaSwap

*Semantic + Kinetic + Dynamic Ontology Layers
with Bitemporal Data & Decision Trace Intelligence*

Version 2.0 | May 2026 | PesaSwap Engineering

References:

- Palantir "Ontology: Finding Meaning in Data" RFX Blog Series (2024)
- Foundation Capital "Context Graphs: AI Trillion-Dollar Opportunity" (2025)
 - Li Ding "On Boosting Semantic Web Data Access" AAAI-05
 - W3C Web Payments Ontology
 - Schema.org Standard Vocabularies

Table of Contents

1. Executive Summary
2. Theoretical Foundations
 - 2.1 Palantir Ontology: Three Layers
 - 2.2 Context Graphs (Foundation Capital)
 - 2.3 Semantic Web & Trust (AAAI-05)
3. PesaSwap Three-Layer Ontology
 - 3.1 Semantic Layer (Object Types)
 - 3.2 Kinetic Layer (Actions & Workflows)
 - 3.3 Dynamic Layer (AI & Simulation)
4. Temporal Architecture (The Clock)
5. Decision Trace Layer
6. Agent Architecture
7. Impact Analysis: Who Benefits
 - 7.1 For Businesses (Merchants)
 - 7.2 For Customers (Diners)
 - 7.3 For Employees (Staff)
 - 7.4 For AI Agents
8. Data Quality & Trust
9. Technical Implementation
10. Phased Delivery Plan
11. Appendix: OWL/RDF Definitions

1. Executive Summary

This document defines a formal ontology for PesaSwap using Palantir's three-layer ontology pattern (Semantic, Kinetic, Dynamic) combined with Foundation Capital's Context Graph thesis and AAAI-05 trust-based data access principles.

The ontology transforms PesaSwap from a payment processor into a "system of agents" where:

- The SEMANTIC layer defines what entities exist (Payments, Customers, Menus, Staff)
- The KINETIC layer tracks what is happening now (live orders, active sessions, real-time state)
- The DYNAMIC layer reasons about what should happen next (AI decisions, predictions, optimizations)

Litmus test: "Can your system tell me what a specific agent knew at 2:14 PM last Tuesday when it made a specific decision?" This transforms PesaSwap from a database into an intelligence platform.

2. Theoretical Foundations

2.1 Palantir Ontology: Three Layers

Palantir's Foundry platform demonstrates that enterprise ontologies work best with three distinct layers, each serving a different purpose:

Semantic Layer - "What exists"

- Defines Object Types: the business entities (Customer, Payment, MenuItem, Table)
- Defines Relationships: how entities connect (Customer places Order, Staff serves Table)
- Defines Properties: attributes on each object (amount, status, timestamp)
- Purpose: Creates shared vocabulary so all systems and people mean the same thing
- Palantir insight: "Make data actionable, discoverable, and governed as business objects"

Kinetic Layer - "What is happening now"

- Defines Actions: operations performed on objects (Approve Refund, Route Payment, Mark Sold-Out)
- Tracks Real-Time State: current orders in progress, active payments, staff on shift
- Maintains Digital Twin: virtual mirror of the physical restaurant (tables, kitchen, bar)
- Purpose: The system maintains parity with the real world at all times

Dynamic Layer - "What should happen next"

- AI Models: predict demand, detect fraud, recommend products
- Simulation: "What if we ran happy hour pricing from 4-6pm?"
- Decision Support: agents recommend actions with confidence scores
- Purpose: Transforms operational awareness into prescriptive intelligence

Key Palantir principle: The ontology is the "semantic layer that brings meaningful business context to raw data. Instead of tables and columns, it models how data represents real-world concepts."

2.2 Context Graphs (Foundation Capital)

Foundation Capital's 2025 thesis: value is shifting from "systems of record" to "systems of agents." The new crown jewel

is the context graph - a living record of decision traces where precedent becomes searchable.

- Adopt standards for basics (Schema.org). Don't reinvent Person, Org, Event.
- Make time first-class. Store what was true WHEN decisions were made.
- Capture reasoning in real time. WHY exceptions happened, not just outcomes.

Every exception = training data. Every override = case study. Every trace = faster future decisions. The graph makes history useful.

2.3 Semantic Web & Trust (AAAI-05)

Li Ding's Web of Belief framework:

- Three interactive worlds: Web, RDF graph world, agent world
- Trust networks determine data reliability before agents reason
- Ontology emergence: structure reveals itself from usage patterns
- Scalability via metadata indexes, not loading everything into memory

3. PesaSwap Three-Layer Ontology

3.1 Semantic Layer (Object Types & Relationships)

Extending Schema.org with PesaSwap domain objects:

Object Type	Extends	Description	Key Properties
Customer	schema:Person	Diner/payer	phone, methods, history
Merchant	schema:Org	Restaurant operator	name, venues, plan, tier
Staff	schema:Person	Employee	role, permissions, schedule
MenuItem	schema:Product	Food/drink item	price, modifiers, category
Menu	schema:ItemList	Item collection	categories, zones, schedule
Table	schema:Place	Dining position	number, zone, capacity
Zone	schema:Place	Venue area	tables, menus, color
Payment	schema:PayAction	Transaction	amount, method, status
Order	schema:Order	Customer order	items, table, total, status
Refund	schema:Transfer	Reversal	amount, reason, approver
Decision	(custom)	Judgment record	actor, reasoning, outcome
Agent	(custom)	AI decision-maker	type, model, confidence

Relationships (typed edges with temporal validity):

Relation	Domain->Range	Description
placesOrder	Customer->Order	Customer initiates order at table
processesPayment	Agent->Payment	AI routes payment method
approvedBy	Decision->Staff	Staff authorized a decision
servedBy	Table->Staff	Which staff served this table
belongsToZone	Table->Zone	Physical zone assignment
pairsWellWith	MenuItem->MenuItem	Upsell/linked product
precedentFor	Decision->Decision	Historical reasoning chain

3.2 Kinetic Layer (Actions & Real-Time State)

Actions are operations triggered by humans or agents on ontology objects:

Action	Target Object	Effect & Business Rule
RoutePayment	Payment	Select method (M-Pesa/card/saved) based on context
ApproveRefund	Refund	Authorize reversal (policy: mgr up to KES 1000)
MarkSoldOut	MenuItem	Toggle availability, notify active orders
AssignZone	Table	Move table to zone, update menu visibility
ActivateMenu	Menu	Toggle menu on/off, affects customer view
OverridePrice	MenuItem	Temporary discount with justification
EscalateComplaint	Order	Flag for manager attention
SplitPayment	Payment	Divide bill among multiple payers

Digital Twin state (maintained in real-time):

- Which tables are occupied/available/reserved
- What orders are in-progress per kitchen station

- Which staff are on shift and their current workload
- What items are close to selling out (consumption rate tracking)

3.3 Dynamic Layer (AI Reasoning & Simulation)

The intelligence layer that turns operational data into prescriptive decisions:

- Payment Agent: "Route to M-Pesa STK (confidence 0.95) based on 12 prior successes"
- Menu Agent: "Pilau demand +40% on rainy days. Suggest increasing prep quantity."
- Service Agent: "Table 7 wait time exceeds 2x average. Alert kitchen."
- Pricing Agent: "Happy hour simulation: 15% price drop = 32% volume increase"
- Fraud Agent: "Card used 5x in 10min. Pattern: card testing. Block + alert."

4. Temporal Architecture (The Clock)

"The hard part isn't the graph. It's the clock." Every fact has bitemporal validity.

```
CREATE TABLE facts (
  id          TEXT PRIMARY KEY,
  entity_id   TEXT NOT NULL,
  entity_type TEXT NOT NULL,
  attribute   TEXT NOT NULL,
  value       JSON NOT NULL,
  valid_from  TIMESTAMP NOT NULL, -- When true in reality
  valid_to    TIMESTAMP,          -- NULL = still true
  recorded_at TIMESTAMP NOT NULL, -- When system learned this
  source      TEXT NOT NULL,      -- provenance
  actor_id    TEXT,
  confidence  REAL DEFAULT 1.0
);
```

Critical queries enabled:

- What was the price when customer was charged? (dispute resolution)
- Who had authority when this refund was approved? (audit)
- What menu was active when the order was placed? (complaint investigation)
- What was the staffing level when service was slow? (root cause analysis)

5. Decision Trace Layer

Captures the "instrumentation of the choice" - WHY decisions were made.

```
Decision Trace = {
  WHO: actor (human or AI), role, authority level
  WHAT: action taken, outcome produced
  WHY: trigger, context snapshot, precedents referenced,
       rules applied, rules overridden, justification text
  WHEN: exact timestamp + what was true at that moment
}
```

Example: "Refund KES 500 approved by shift-manager James because kitchen understaffed (2/4 staff), wait time 2x average, customer is loyal (8 visits, KES 15K lifetime). Precedent: 3 similar approvals this month."

6. Agent Architecture

Four AI agents operate within the kinetic+dynamic layers:

Payment Decision Agent

- Routes to optimal method based on customer graph + context
- Assesses fraud risk using temporal patterns
- Records decision trace with full reasoning

Menu Intelligence Agent

- Discovers demand patterns (weather, time, events)
- Optimizes category ordering from conversion data
- Predicts sold-out risk; suggests prep quantities

Service Quality Agent

- Monitors wait times; alerts on anomalies
- Correlates staff with satisfaction outcomes
- Predicts complaint risk before customer complains

Fraud & Anomaly Agent

- Detects card testing, refund collusion, split gaming
- Records all flags with evidence and confidence scores

7. Impact Analysis: Who Benefits

The ontology-enhanced merchantApp creates value for four distinct personas. Each benefits differently from the three ontology layers.

7.1 For Businesses (Merchants/Restaurant Owners)

Revenue Growth

- AI-powered upselling: linked products from actual purchase graph increase AOV 15-25%
- Dynamic menu optimization: categories auto-ordered by conversion rate
- Demand prediction: prepare the right quantities, reduce waste by 20-30%
- Zone-based menus: premium areas show premium items, bar shows drinks-only

Operational Efficiency

- Digital twin of restaurant: real-time view of all tables, orders, kitchen load
- Staff performance correlation: know which staff assignments maximize revenue
- Automated scheduling: menus activate/deactivate by time without manual intervention
- Complaint prediction: intervene before customer complains (long wait + understaffed)

Risk Reduction

- Fraud detection: catch card testing, refund collusion, split gaming automatically
- Full audit trail: answer "who approved what and why" for any historical decision
- Dispute resolution in seconds: bitemporal queries prove what price/menu was active
- Compliance: every action has provenance, actor, timestamp, justification

Strategic Intelligence

- Searchable precedent: "How did we handle similar situations before?"
- Simulation: "What if we extend happy hour to 7pm?" with predicted revenue impact
- Competitive insight: understand what drives YOUR customers (not industry averages)
- Compounding value: system gets smarter with every transaction (not resets to zero)

7.2 For Customers (Diners)

Faster Service

- One-tap payments: agent routes to best method (no form filling for returning customers)
- M-Pesa STK push: zero UI for mobile money (just confirm on phone)
- Smart split: AI remembers how this group usually splits (equal, by item, by person)

Personalized Experience

- Menu in their language: auto-detected from phone, 4 languages supported
- Remembered preferences: "Sarah always orders decaf after 6pm, prefers window seat"
- "Goes well with..." suggestions based on what similar customers loved

PesaSwap - Ontology for Payments and Merchant Applications

- Allergen safety: dietary tags + modifiers ensure safe customization

Transparency & Trust

- Real-time order tracking: know when food is being prepared, ready, served
- Clear pricing: modifiers show exact price impact before ordering
- Instant receipts: digital receipt with full breakdown, shareable
- Fair dispute resolution: system has full temporal record of what happened

Reduced Friction

- No app download required: QR scan opens web experience instantly
- No account required: pay with phone number (M-Pesa) or card
- Sold-out items clearly marked: no ordering then being told "sorry, unavailable"
- Multiple payment methods: M-Pesa, card, wallet - whatever they prefer

7.3 For Employees (Staff)

Empowerment & Autonomy

- Clear authority: system knows what each role can approve (no guessing)
- Precedent access: "3 similar refunds were approved this month" supports decisions
- Reduced decision anxiety: AI recommends action with confidence score
- Override with justification: staff can override AI if they capture WHY

Performance Visibility

- Fair attribution: system tracks which staff correlates with better outcomes
- Tip transparency: staff see their tip rates vs. averages (motivation)
- Skill development: identify what top performers do differently
- Workload balancing: AI detects overload and suggests rebalancing

Operational Support

- Kitchen alerts: system predicts rush before it happens (prep accordingly)
- Order routing: items auto-route to correct station (kitchen vs bar)
- Sold-out management: one tap to 86 an item, customers notified instantly
- Reservation awareness: know who is coming, their preferences, party size

Reduced Stress

- AI handles routine decisions (payment routing, menu filtering)
- Complaints predicted early: intervene with a water refill before complaint escalates
- No blame culture: temporal record shows context (understaffed, rush hour)
- Training from precedent: new staff learn from searchable past decisions

7.4 For AI Agents

Rich Context for Better Decisions

- Full customer graph: 47 payments, 0.98 success rate, prefers M-Pesa, no fraud flags
- Temporal context: time of day, day of week, venue load, weather
- Operational state: kitchen capacity, staff count, current wait times
- Historical precedent: "In 5 similar situations, outcome was X"

Continuous Learning

- Every decision becomes training data (outcome tracked against prediction)
- Emergent patterns surface automatically (no manual feature engineering)
- Trust scores improve over time (agents learn which sources are reliable)
- Confidence calibration: agents know when to decide vs. escalate to human

Clear Accountability

PesaSwap - Ontology for Payments and Merchant Applications

- Every agent decision has a trace (auditable, explainable)
- Confidence scores make uncertainty explicit (no black-box decisions)
- Human override captured: agent learns from corrections
- Bounded authority: agents cannot exceed their defined action scope

Collaborative Intelligence

- Agents share context graph (Payment Agent informs Fraud Agent informs Service Agent)
- Cross-domain reasoning: "High fraud risk + first-time customer + large order = extra verification"
- Escalation protocol: agents know when to hand off to humans
- Multi-agent coordination: Menu Agent + Service Agent = optimal dining experience

8. Data Quality & Trust

From AAAI-05: not all data sources are equal. Agents must evaluate reliability.

Trust Scores:

POS System (direct sync)	: 0.95
Staff Manual Entry	: 0.85
Customer Self-Report	: 0.70
AI Agent Inference	: 0.60-0.90
Third-Party API	: 0.80

Quality Dimensions:

ACCURACY	- Validated against ground truth
COMPLETENESS	- All required attributes present
TIMELINESS	- Current or stale?
CONSISTENCY	- Contradicts other facts?
PROVENANCE	- Traceable origin?
TRUST	- Reliable source?

9. Technical Implementation

Layer	Technology	Purpose
Frontend	React 19 + TanStack	Already built, proven at scale
Edge	Cloudflare Workers	Global, <10ms, auto-scale
Graph	SurrealDB	Entity relations, traversal, patterns
Temporal	Cloudflare D1	Bitemporal facts, decisions, audit
Vector	CF Vectorize	Semantic precedent search
AI	Workers AI + Claude	Agent reasoning, discovery
Auth	Clerk	Multi-tenant, RBAC, SSO
Payments	PesaSwap SDK	M-Pesa, cards, wallets
Storage	Cloudflare R2	Images, PDFs, media
Queue	CF Queues	Async trace writes, events

10. Phased Delivery Plan

Phase 1: Semantic + Temporal Foundation (Weeks 1-2)

- Deploy D1 bitemporal schema (entities, facts, decisions, relationships)
- Define all Object Types and Relationships in code
- Temporal middleware: every mutation records provenance + timestamps
- Migrate localStorage to D1; implement `getEntityStateAt(id, timestamp)`
- Auth (Clerk) with multi-tenant isolation

Phase 2: Kinetic Layer + Decisions (Weeks 3-4)

- Define all Actions with business rules and authorization
- Build digital twin: real-time restaurant state
- Decision trace capture middleware (intercepts all mutations)
- Staff reasoning UI (refund reason, override justification)
- Payment Agent v1: routes based on customer history graph

Phase 3: Dynamic Layer + Intelligence (Weeks 5-6)

- Deploy SurrealDB for graph queries and pattern discovery
- Menu Intelligence Agent: co-purchase patterns, demand prediction
- Service Quality Agent: staff-outcome correlations
- Precedent search API (structured + vector similarity)
- Dashboard "AI Insights" page with discovered patterns

Phase 4: Trust + Enterprise (Weeks 7-8)

- Trust scoring for all data sources
- Fraud Agent: anomaly detection on payments/refunds
- Provenance explorer UI (trace any fact to origin)
- Simulation engine: "What if?" scenarios with predicted outcomes
- Full audit replay: "Show me everything at 2:14 PM last Tuesday"

TOTAL: 8 weeks (2 engineers) | COST: \$80-215/month at 100K users

Each phase delivers working features. The ontology compounds: Phase 4 intelligence is impossible without Phase 1 temporal data.

11. Appendix: OWL/RDF Definitions

```

@prefix ps: <https://ontology.pesaswap.io/> .
@prefix schema: <https://schema.org/> .
@prefix owl: <http://www.w3.org/2002/07/owl#> .

# Semantic Layer - Object Types
ps:Customer a owl:Class ; rdfs:subClassOf schema:Person .
ps:Merchant a owl:Class ; rdfs:subClassOf schema:Organization .
ps:Staff a owl:Class ; rdfs:subClassOf schema:Person .
ps:MenuItem a owl:Class ; rdfs:subClassOf schema:Product .
ps:Payment a owl:Class ; rdfs:subClassOf schema:PayAction .
ps:Order a owl:Class ; rdfs:subClassOf schema:Order .
ps:Decision a owl:Class ; rdfs:subClassOf schema:Action .
ps:Agent a owl:Class ; rdfs:subClassOf schema:SoftwareApplication .

# Temporal Properties
ps:validFrom a owl:DatatypeProperty ; rdfs:range xsd:dateTime .
ps:validTo a owl:DatatypeProperty ; rdfs:range xsd:dateTime .
ps:recordedAt a owl:DatatypeProperty ; rdfs:range xsd:dateTime .
ps:confidence a owl:DatatypeProperty ; rdfs:range xsd:float .

# Kinetic Layer - Actions
ps:RoutePayment a owl:Class ; rdfs:subClassOf schema:Action .
ps:ApproveRefund a owl:Class ; rdfs:subClassOf schema:Action .
ps:MarkSoldOut a owl:Class ; rdfs:subClassOf schema:Action .
ps:OverridePrice a owl:Class ; rdfs:subClassOf schema:Action .

# Dynamic Layer - Agent Capabilities
ps:predicts a owl:ObjectProperty ;
  rdfs:domain ps:Agent ; rdfs:range ps:MenuItem .
ps:recommends a owl:ObjectProperty ;
  rdfs:domain ps:Agent ; rdfs:range ps:Decision .
ps:detects a owl:ObjectProperty ;
  rdfs:domain ps:Agent ; rdfs:range ps:Anomaly .

# Relationships
ps:placesOrder a owl:ObjectProperty ;
  rdfs:domain ps:Customer ; rdfs:range ps:Order .
ps:processesPayment a owl:ObjectProperty ;
  rdfs:domain ps:Agent ; rdfs:range ps:Payment .
ps:precedentFor a owl:ObjectProperty ;
  rdfs:domain ps:Decision ; rdfs:range ps:Decision .

```

This ontology follows Palantir's principle: "The ontology is the semantic layer that brings meaningful business context to raw data." It is designed to be extended as agents discover new patterns. The bitemporal model ensures evolution is tracked.